

```

#!/usr/bin/env bash
#
=====
=====
# OMNIBUS ARTIFACT GENERATOR: TOPOLOGICALLY INSULATED INVARIANT SYSTEMS
# Framework: Invariant-Logic-Engine OMNIBUS v3.0
# Combined Deliverable: Fused Framework (With Precedent) & Standalone Core Substrate
#
=====
=====
set -euo pipefail

INFO="\033[0;34mINFO\033[0m"
SUCCESS="\033[0;32mSUCCESS\033[0m"
WARNING="\033[0;33mWARNING\033[0m"

echo -e "${INFO} Initializing Combined Academic & Production Workspace..."

# Structural Layout Scaffolding
mkdir -p workspace/fused_framework/src/math
mkdir -p workspace/fused_framework/config
mkdir -p workspace/standalone_core/src/math
mkdir -p workspace/standalone_core/config

#
=====
=====
# LANE 1: THE FUSED FRAMEWORK (WITH JOHN'S MATH PRECEDENT INTEGRATED)
#
=====
=====
echo -e "${INFO} Generating Lane 1: Fused System (with 80D Aperiodic Precedent)..."

cat << 'EOF' > workspace/fused_framework/src/math/manifold_fused.tex
\documentclass[10pt,journal,compsoc]{IEEEtran}
\usepackage{amsmath}
\usepackage{amssymb}
\usepackage{amsthm}
\usepackage{bm}
\begin{document}
\title{Topologically Insulated Invariant Systems: Fused High-Dimensional Framework}
\author{Borealis S. Hedling, Senior Member, IEEE}
\maketitle
\begin{abstract}

```

This version explicitly builds upon higher-dimensional linear projection kernels and formal verification frameworks established in contemporary aperiodic geometric mechanics. By referencing 4D-to-3D projection spaces, we structurally anchor an 80D continuous soft Riemannian manifold tracking engine.

`\end{abstract}`

`\section{Mathematical Invariance}`

Let $(\mathcal{M}_{80}, g)$ be a smooth, continuous, 80-dimensional soft Riemannian manifold embedded in a higher ambient informational space $\mathbb{R}^D$ where $80 \ll D$. State metrics map onto a complex density matrix $\rho \in \mathcal{H}_{80 \times 80}$ bound to CPTP Kraus channels to enforce $\text{Tr}(\rho) = 1.0$. Proving linear injectivity via higher-dimensional projection operators establishes that continuous holonomy maps preserve structural trace invariants perfectly without coordinate collapse.

`\end{document}`

EOF

```
cat << 'EOF' > workspace/fused_framework/src/lib.rs
```

```
use pyo3::prelude::*;
```

```
use pyo3::exceptions::PyValueError;
```

```
use num_complex::Complex64;
```

```
use nalgebra::DMatrix;
```

```
#[pyclass]
```

```
pub struct FusedManifold80DAdapter { pub drift_tolerance: f64 }
```

```
#[pymethods]
```

```
impl FusedManifold80DAdapter {
```

```
    #[new] #[pyo3(signature = (drift_tolerance=1e-7))]
```

```
    fn new(drift_tolerance: f64) -> Self { Self { drift_tolerance } }
```

```
    fn normalize_density_80d(&self, py_matrix: Vec<Vec<(f64, f64)>>) ->
```

```
PyResult<Vec<Vec<(f64, f64)>>> {
```

```
    let mut matrix = self.parse_py_matrix(py_matrix)?;
```

```
    let rho_dagger = matrix.adjoint();
```

```
    matrix = (&matrix + &rho_dagger) * 0.5;
```

```
    let trace: Complex64 = matrix.diagonal().iter().sum();
```

```
    if trace.re <= 0.0 { return Err(PyValueError::new_err("80D trace mass evaporated.")); }
```

```
    matrix = matrix.map(|val| val / Complex64::new(trace.re, 0.0));
```

```
    Ok(self.export_to_py(matrix))
```

```
}
```

```
}
```

```
impl FusedManifold80DAdapter {
```

```
    fn parse_py_matrix(&self, source: Vec<Vec<(f64, f64)>>) ->
```

```
PyResult<DMatrix<Complex64>> {
```

```

    let nrows = source.len();
    let mut matrix = DMatrix::from_element(nrows, nrows, Complex64::new(0.0, 0.0));
    for i in 0..nrows {
        for j in 0..nrows {
            let (re, im) = source[i][j]; matrix[(i, j)] = Complex64::new(re, im);
        }
    }
    Ok(matrix)
}
fn export_to_py(&self, source: DMatrix<Complex64>) -> Vec<Vec<(f64, f64)>> {
    let mut out = vec![vec![(0.0, 0.0); source.ncols()]; source.nrows()];
    for i in 0..source.nrows() {
        for j in 0..source.ncols() { out[i][j] = (source[(i, j)].re, source[(i, j)].im); }
    }
    out
}
}

```

```

#[pymodule]
fn invariant_logic_adapter(_py: Python, m: &PyModule) -> PyResult<()> {
    m.add_class::<FusedManifold80DAdapter>()?; Ok(())
}
EOF

```

```

cat << 'EOF' > workspace/fused_framework/Cargo.toml
[package]
name = "invariant_logic_adapter"
version = "2.5.0"
edition = "2021"

```

```

[lib]
name = "invariant_logic_adapter"
crate-type = ["cdylib"]

```

```

[dependencies]
num-complex = "0.4"
nalgebra = "0.32"
pyo3 = { version = "0.20", features = ["extension-module"] }
EOF

```

```

#
=====
=====

```

```
# LANE 2: THE STANDALONE ENGINE SUBSTRATE (COMPLETELY CLEAN CORE
ENGINE)
```

```
#
```

```
=====
=====
```

```
echo -e "${INFO} Generating Lane 2: Standalone Pure Production Substrate..."
```

```
cat << 'EOF' > workspace/standalone_core/src/math/independent_core.tex
\documentclass[11pt,preview,varwidth]{standalone}
\usepackage{amsmath}
\usepackage{amssymb}
\usepackage{amsthm}
\usepackage{bm}
\begin{document}
\section*{Topologically Insulated Invariant Systems: Isolated Production Kernel}
Let  $(M, g)$  be a smooth, connected,  $d$ -dimensional Riemannian manifold
embedded in  $\mathbb{R}^D$  ( $d \ll D$ ). We encapsulate system state updates purely within a
trace-class-1 positive semi-definite density matrix  $\rho$  in  $\mathcal{H}_{128}$  bound strictly to
Kraus operational dynamics ( $\text{Tr}(\rho) = 1.0$ ). Data changes or system modifications
manifest directly as explicit non-local geometric distortions bulged across alternative sectors of
the tracking matrix.
\end{document}
EOF
```

```
cat << 'EOF' > workspace/standalone_core/src/lib.rs
```

```
use pyo3::prelude::*;
use pyo3::exceptions::PyValueError;
use num_complex::Complex64;
use nalgebra::{DMatrix, Vector4};
```

```
#[pyclass]
```

```
pub struct PureQuantumProxyAdapter { pub altitude_threshold: f64 }
```

```
#[pymethods]
```

```
impl PureQuantumProxyAdapter {
    #[new] #[pyo3(signature = (altitude_threshold=1.0))]
    fn new(altitude_threshold: f64) -> Self { Self { altitude_threshold } }
```

```
    fn normalize_density_matrix(&self, py_matrix: Vec<Vec<(f64, f64)>>) ->
PyResult<Vec<Vec<(f64, f64)>>> {
    let mut matrix = self.parse_py_matrix(py_matrix)?;
    let rho_dagger = matrix.adjoint();
    matrix = (&matrix + &rho_dagger) * 0.5;
    let trace: Complex64 = matrix.diagonal().iter().sum();
```

```

        if trace.re <= 0.0 { return Err(PyValueError::new_err("Non-positive trace mass.")); }
        matrix = matrix.map(|val| val / Complex64::new(trace.re, 0.0));
        Ok(self.export_to_py(matrix))
    }

    fn inverse_exponential_map_projection(&self, py_vector: Vec<f64>) -> PyResult<Vec<f64>> {
        if py_vector.len() != 4 { return Err(PyValueError::new_err("Vector must contain exactly 4
components.")); }
        let mut tangent_vector = Vector4::new(py_vector[0], py_vector[1], py_vector[2],
py_vector[3]);
        let norm = tangent_vector.norm();
        if norm > 1e-9 { tangent_vector *= norm.atanh() / norm; }
        Ok(vec![tangent_vector[0], tangent_vector[1], tangent_vector[2], tangent_vector[3]])
    }
}

impl PureQuantumProxyAdapter {
    fn parse_py_matrix(&self, source: Vec<Vec<(f64, f64)>>) ->
PyResult<DMatrix<Complex64>> {
        let nrows = source.len();
        let mut matrix = DMatrix::from_element(nrows, nrows, Complex64::new(0.0, 0.0));
        for i in 0..nrows {
            for j in 0..nrows {
                let (re, im) = source[i][j]; matrix[(i, j)] = Complex64::new(re, im);
            }
        }
        Ok(matrix)
    }

    fn export_to_py(&self, source: DMatrix<Complex64>) -> Vec<Vec<(f64, f64)>> {
        let mut out = vec![vec![(0.0, 0.0); source.ncols()]; source.nrows()];
        for i in 0..source.nrows() {
            for j in 0..source.ncols() { out[i][j] = (source[(i, j)].re, source[(i, j)].im); }
        }
        out
    }
}

#[pymodule]
fn invariant_logic_adapter(_py: Python, m: &PyModule) -> PyResult<()> {
    m.add_class::<PureQuantumProxyAdapter>()?; Ok(())
}
EOF

```

```
cat << 'EOF' > workspace/standalone_core/Cargo.toml
```

```
[package]
name = "invariant_logic_adapter"
version = "2.2.0"
edition = "2021"
```

```
[lib]
name = "invariant_logic_adapter"
crate-type = ["cdylib"]
```

```
[dependencies]
num-complex = "0.4"
nalgebra = "0.32"
pyo3 = { version = "0.20", features = ["extension-module"] }
EOF
```

```
#
=====
=====
# AUTOMATED WORKSPACE ENGINE VERIFICATION
#
=====
=====
echo -e "\n-----"
echo -e "${INFO} Triggering isolated runtime toolchain builds..."

if command -v maturin &> /dev/null; then
    echo -e "${INFO} Maturin found. Building Lane 1 (Fused Engine)..."
    (cd workspace/fused_framework && maturin develop --release)
    echo -e "${INFO} Building Lane 2 (Standalone Core)..."
    (cd workspace/standalone_core && maturin develop --release)
    echo -e "${SUCCESS} Both native FFI binaries compiled successfully."
else
    echo -e "${WARNING} maturin toolchain not found in path. Outputting raw source directory
modules only."
fi

echo -e
"\n=====
echo -e "${SUCCESS} INTEGRATED ARTIFACT OMNIBUS PACKAGED SECURELY IN
/workspace"
echo -e
"=====\\n"
```